

CPU Scheduling Concepts and Algorithms

Prof. Mehdi Pirahandeh

1 Introduction to CPU Scheduling

In operating systems, CPU scheduling determines which processes in the ready queue are allocated to the CPU for execution. Scheduling aims to ensure efficient CPU utilization and minimize waiting times. Several algorithms exist, each with different benefits and drawbacks.

2 Basic Concepts

- **Multiprogramming:** A technique that aims to maximize CPU utilization by allowing multiple processes to be executed simultaneously.
- **CPU-I/O Burst Cycle:** Processes alternate between CPU execution and I/O wait. Efficient scheduling requires an understanding of CPU burst distribution, which consists of many short bursts and fewer long ones.

3 CPU Scheduling Algorithms

There are various CPU scheduling algorithms, each with its characteristics:

3.1 First-Come, First-Served (FCFS)

- **Description:** Processes are executed in the order they arrive.
- **Advantages:** Simple to implement.
- **Disadvantages:** Prone to the *convoy effect*, where short processes wait behind long ones, leading to high average waiting time.

3.2 Shortest Job First (SJF)

- **Description:** The process with the shortest CPU burst is executed first.
- **Advantages:** Minimizes average waiting time.
- **Disadvantages:** Requires burst length prediction, which may not be accurate.

3.3 Shortest Remaining Time First (SRTF)

- **Description:** A preemptive version of SJF where the CPU can be allocated to a new process if it has a shorter remaining time than the current one.
- **Advantages:** Minimizes waiting time further than SJF.
- **Disadvantages:** High overhead due to process preemption.

3.4 Round Robin (RR)

- **Description:** Each process gets a fixed time slice (quantum) and, after expiration, is placed at the end of the queue.
- **Advantages:** Fair scheduling with good response time.
- **Disadvantages:** High turnaround time for small quantum values.

3.5 Priority Scheduling

- **Description:** Processes are executed based on priority. Both preemptive and non-preemptive versions exist.
- **Advantages:** Flexible with priority handling.
- **Disadvantages:** Lower-priority processes may suffer from starvation. Aging can be used to prevent starvation.

3.6 Multilevel Queue Scheduling

- **Description:** The ready queue is divided into multiple queues based on priority levels, and each queue has its own scheduling algorithm.
- **Advantages:** Efficient handling of processes with different priorities.
- **Disadvantages:** Complexity in configuration.

3.7 Multilevel Feedback Queue Scheduling

- **Description:** Processes can move between different priority queues depending on their behavior.
- **Advantages:** Adaptable and dynamic.
- **Disadvantages:** More complex to configure than other algorithms.

4 Thread and Multi-Processor Scheduling

- **Process Contention Scope (PCS) vs System Contention Scope (SCS):** PCS schedules threads within a process, while SCS schedules system-wide.
- **Multiple-Processor Scheduling:** A strategy that balances load across multiple CPUs. Techniques include *push migration* (moving tasks from busy CPUs to idle ones) and *pull migration* (idle CPUs pull tasks from busy ones).
- **Processor Affinity:** A thread's preference for certain processors to exploit cache memory. It can be soft (no strict guarantees) or hard (restricts the thread to a set of CPUs).
- **NUMA Scheduling:** Accounts for memory access times in multi-processor systems to optimize performance.

5 Real-Time Scheduling

5.1 Soft Real-Time Systems

Critical tasks get higher priority, but deadlines are not guaranteed.

5.2 Hard Real-Time Systems

Tasks must meet their deadlines, requiring strict scheduling.

5.3 Rate Monotonic Scheduling (RMS)

A fixed-priority algorithm where shorter intervals receive higher priority.

5.4 Earliest Deadline First (EDF)

A dynamic-priority algorithm where processes with the earliest deadlines are prioritized.

6 Scheduling Algorithm Evaluation

CPU scheduling algorithms can be evaluated using methods like:

- **Deterministic Modeling:** A method of predicting performance based on fixed workloads.
- **Queueing Models:** Probabilistic models used to predict CPU and I/O burst behavior.
- **Simulation:** Simulations can be used to gather performance statistics and evaluate different scheduling algorithms.

Algorithm	Preemptive/Non-Preemptive	Advantages	Disadvantages	Example
FCFS	Non-Preemptive	Simple to implement	Convoy effect; high waiting time	Avg. waiting time = 17 ms
SJF	Non-Preemptive	Minimum average waiting time	Requires accurate burst prediction	Avg. waiting time = 7 ms
SRTF	Preemptive	Minimizes waiting time	Overhead from preemption	Avg. waiting time = 6.5 ms
Round Robin (RR)	Preemptive	Fair and responsive	High turnaround time with small quantum	Avg. waiting time = 22 ms
Priority Scheduling	Preemptive/Non-Preemptive	Flexible	Starvation; aging required	Avg. waiting time = 8.2 ms
Multilevel Feedback Queue	Preemptive	Dynamic and adaptable	Complex to configure	Uses FCFS in last queue

Table 1: Comparison of CPU Scheduling Algorithms

7 Comparison of Scheduling Algorithms

8 Graphical Representation of Results

The following figure displays the results of scheduling algorithms in terms of average waiting time, turnaround time, and CPU utilization.

9 Conclusion

Each scheduling algorithm provides different benefits, and the choice of algorithm depends on the system requirements, such as waiting time, turnaround time, and CPU efficiency. Algorithms like SJF and SRTF minimize waiting time, while Round Robin ensures fairness. More complex algorithms, such as Multilevel Feedback Queues, adapt dynamically to process behavior but are harder to configure.

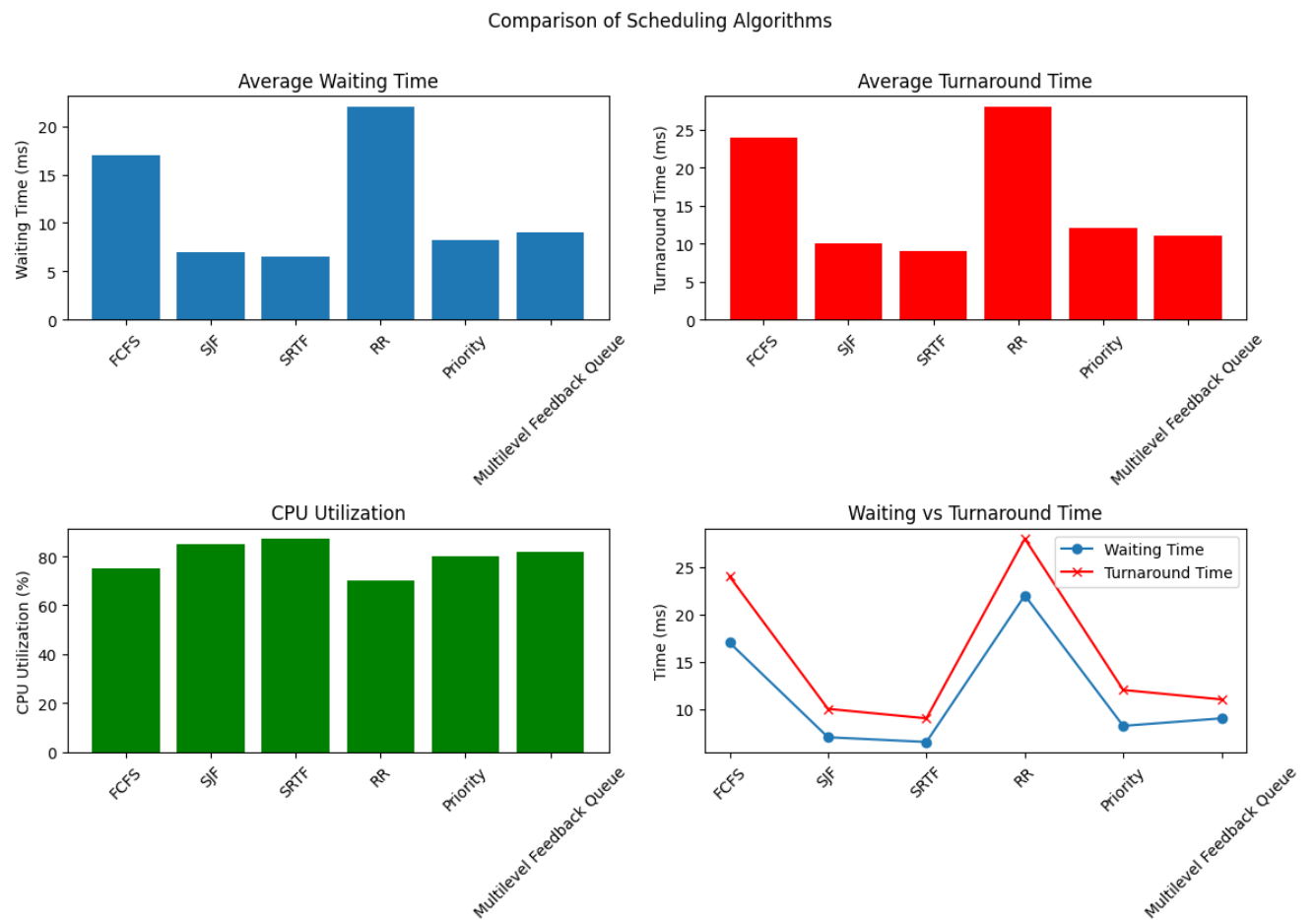


Figure 1: Comparison of CPU Scheduling Algorithms (Waiting Time, Turnaround Time, and CPU Utilization)